

AI Ops Module 1 Assignment

Name: Sri Harsha V Roll Number: DA24B034

Question 1 – Technical Debt Diagnosis

1.

(a) **Entanglement (CACE)**

Changing the rounding logic of the “estimated delivery time” feature affected the unrelated “favorite restaurants” feature. This shows that the two parts are tightly coupled.

(b) **Undeclared Consumers.**

The marketing dashboard was accessing the model’s raw output table without the ML team’s knowledge. This creates an undeclared dependency, making changes to the model’s output potentially affect downstream users that are not known to the ML team.

(c) **Pipeline Jungles / Glue Code.**

The training process uses 14 undocumented shell scripts with no orchestration. This makes the pipeline difficult to understand, reproduce, and modify safely.

2.

Mitigation : For (c), I would use an **MLflow Project** to organize the training workflow. The **MLproject** file can define the entry point and parameters, while the environment specification makes the setup reproducible. This makes the workflow easier to reproduce and maintain.

Question 2 - Written Analysis

Across the 7 MLP runs on MNIST, the best-performing configuration is **mlp-h128-lr001-b64** (`hidden_layer_sizes=(128,)`, `learning_rate_init=0.001`, `batch_size=64`), achieving the highest validation accuracy (0.979) and `val_f1_macro` (0.979), despite converging in only 21 iterations. This suggests a single wide hidden layer with a low learning rate generalizes best on this task, and that additional depth (e.g., 128×128 or 128×64) or a larger learning rate does not meaningfully improve results.

Regarding overfitting: the run with the lowest **train_loss** (**mlp-h128x64-lr001-b128**, `train_loss=0.014`) does not produce the highest **val_accuracy** (0.978, marginally below the best run). Since a lower training loss does not translate into a proportionally better validation score, this run shows mild signs of overfitting—the model is fitting the training data slightly more aggressively without a corresponding generalization gain. Conversely, **mlp-h64-lr01-b64** has both high **train_loss** (0.061) and the lowest **val_accuracy** (0.966), indicating underfitting rather than overfitting.

Comparing hyperparameters, **learning rate** has the larger and more consistent effect: for every matched architecture, dropping the learning rate from 0.01 to 0.001 improved **val_accuracy** (e.g., +0.011 for the single-128 architecture, +0.006 for 64, +0.004 for 128×64). Architecture size (number/width of hidden layers) produced smaller, less consistent swings (~ 0.007 – 0.008), making learning rate the more influential hyperparameter here.